# AdventureWorks Analytics Project

Week 3 Friday Hackathon · Project Documentation · Prepared by Aqsa Saqib

## 1. What is this project about?

AdventureWorks is a company that makes and sells bicycles, bike parts, clothing and accessories. Its database is very good at **saving daily orders**, but it is hard to use for **reports**. The data is spread over 68 tables, so even a simple question like "how much did we sell last month?" needs a long query.

So I built an **analytics layer**: a separate set of ready-made tables and views (a schema called `analytics`). They already contain the answers in a clean form. Any report or dashboard can now read from them, and nobody has to touch the raw tables again. After that, I used Python to make **13 charts** and wrote **business recommendations**.

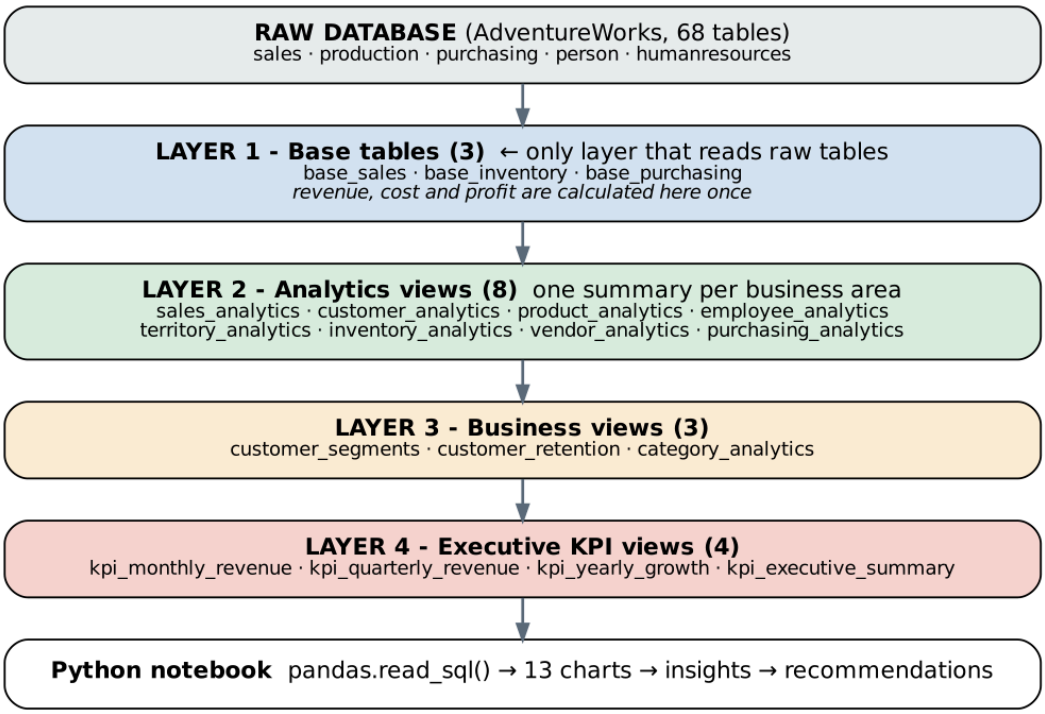
## 2. The dataset

The company sells in two ways: **Online** (people buy on the website) and through **Resellers** (bike shops buy in bulk through salespeople). The data covers **30 May 2022 to 29 June 2025**. The 68 tables are grouped into 5 schemas:

| Schema | What is inside | Main tables I used |
|---|---|---|
| `sales` | orders, customers, shops, salespeople, regions | salesorderheader (31,465 orders), salesorderdetail (121,317 order lines), customer, store, salesterritory |
| `production` | products, categories, stock, product cost | product (504), productinventory, productcosthistory |
| `purchasing` | suppliers and what we buy from them | vendor (104), purchaseorderheader, purchaseorderdetail |
| `person`, `humanresources` | names of people, employees | person, employee |

## 3. How I built it: 4 layers

I built **18 tables and views in 4 layers**. Each layer only reads from the layer above it, so the raw tables are read only once, in Layer 1.




Layer 1 = clean copy of the raw data · Layer 2 = one summary per business area · Layer 3 = customer groups, retention and categories · Layer 4 = the final numbers for management.

## 4. SQL skills I used

| SQL skill | Where I used it (in simple words) |
|---|---|
| CTEs (`WITH`) | To build `base_sales` step by step: first the order lines, then product and cost, then the names. |
| Window functions | `LAG` to compare with last month, last quarter and last year; `SUM() OVER` for running totals. |
| Ranking | `RANK` for best-selling products, `DENSE_RANK` inside each category, `NTILE(4)` to split customers into 4 groups. |
| `CASE WHEN` | For every label: customer group, stock status, supplier rating, profit status. |
| Conditional aggregation | To split revenue into online and reseller in one query. |
| Complex joins | 10 tables joined in `base_sales`, product cost matched by date, and a self-join for customer retention. |

## 5. Problems I faced and how I solved them

| Problem | What I did |
|---|---|
| The Postgres version has no "LineTotal" column (price of each order line). | I calculated it myself: quantity × price × (1 − discount). It matches the order totals exactly ($109,846,381.40). |
| The first and last months are not complete (shop orders stop after April 2025). | I used only full months (June 2022 to April 2025) for growth, so the charts do not show a fake drop at the end. |
| 2022 and 2025 are not full years. | I also compared every year on January to April only. |
| Product cost changes every year. | Each sale uses the cost that was valid on its order date. |
| Shops spend much more than single buyers, and salespeople joined at different times. | I grouped shops and single buyers separately, and compared salespeople on sales per month. |

## 6. Things I assumed

- **Revenue** = order amount without tax and shipping. **Profit** = revenue minus product cost.
- **"Today"** = the last order date in the data (29 June 2025).
- Purchase orders that were *Rejected* are not counted as spending.
- The stock table shows today's stock. Retention for 2024 → 2025 is not final yet (2025 is not over).

## 7. What I found

- **The business is growing fast:** +38% in 2024, and +74% in January-April 2025 compared with the same months of 2024.
- **Online sales make almost all the profit:** about 41% profit online, but about 0% from shops (below zero in 2024).
- **A few customers matter most:** 13% of customers bring 59% of the money, and 146 shops worth $12.4M stopped ordering.
- **Some products lose money:** Mountain-200 bikes sell best, but 32 products lose $646K in total (mostly Touring bikes and jerseys).
- **Stock and buying problems:** 46 products that customers buy are out of stock, and in early 2025 the company spent more on buying ($13.6M) than it earned from sales ($12.8M).

## 8. My recommendations

1. Set a minimum profit on shop (reseller) prices, and re-price or stop selling the 32 loss-making products.
2. Restock the 46 out-of-stock best-sellers first.
3. Call back the 146 lost shops, and send new online buyers an offer so they buy again.
4. Let the top salespeople coach the 5 weaker ones. Focus more on Europe and Australia (fastest growth).
5. Buy based on what really sells, and use less of the 11 suppliers whose items often get rejected.

> **How to run it:** (1) load the data with `install.sql`, (2) run `analytics_pipeline.sql` (it ends with 4 OK checks), (3) open `executive_analysis.ipynb`, type your password and click Run All. On Google Colab, use `executive_analysis_colab.ipynb`; it does steps 1 and 2 for you.